Author: Tianqi Shen & Ziye Ma & GPT-4o

Date: 14 November 2025

Numerical Experiments of Tensor Over-parametrization (Simulated Version)

Basic Example

To simulate the effect of tensor over-parameterization in escaping spurious local minima that arise in the matrix space, we consider the following instance of the Matrix Sensing (MS) problem:
Let the ground truth matrix be

$$M^* = zz^\top = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

where $z = [1, 0]^\top$. The linear measurement operator $\mathcal{A} : \mathbb{R}^{2\times 2} \to \mathbb{R}^3$ is defined through the sensing matrices:

$$A_1 = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{2} \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0 & \frac{\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2} & 0 \end{bmatrix}, \quad A_3 = \begin{bmatrix} 0 & 0 \\ 0 & \frac{\sqrt{3}}{2} \end{bmatrix}.$$

We consider the following objective function:

$$h(x) = f(xx^\top) = \frac{1}{2}\|\mathcal{A}(xx^\top - zz^\top)\|_2^2,$$

where $x \in \mathbb{R}^2$ is the optimization variable (we still use $X \in \mathbb{R}^{2\times 1}$ later for consistency). The operator $\mathcal{A}$ satisfies the Restricted Isometry Property (RIP) of order 2 with constant $\frac{1}{2}$.
Let

$$\hat{x} = [0, \frac{1}{\sqrt{2}}]^\top.$$

As shown in the manuscript, $\hat{x}$ is a spurious local minimum of the objective $h(x)$.

In [1]:
```
%matplotlib inline
#import matplotlib_inline   # setup output image format, do not use 'svg' if need 'pdf'
#matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 300
import matplotlib
import argparse
import numpy as np
import jax
import jax.numpy as jnp
from itertools import chain
from tqdm import tqdm
import os
os.environ['CUDA_VISIBLE_DEVICES'] = "3"
```

Functional Helpers for Matrix Computation

- Apply the linear measurement operator $\mathcal{A}$ (*func:* `apply_A`):

$$\mathcal{A}(M) = [\,\langle A_1, M\rangle, \langle A_2, M\rangle, \cdots, \langle A_m, M\rangle\,]^\top \in \mathbb{R}^m.$$

- Apply the adjoint operator $\mathcal{A}^*$ (*func:* `adjoint_A`):

$$\mathcal{A}^*(y) = \sum_{i=1}^{m} y_i \cdot A_i \in \mathbb{R}^{n\times n}.$$

- Compute the smallest eigenvalue λ_n and eigenvector u_n of a symmetric matrix G (*func:* `min_eigpair_symmetric`):

$$G = \sum_{\theta=1}^{n} \lambda_\theta u_\theta u_\theta^\top, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n.$$

- Compute the smallest nonzero singular value σ_r and associated vectors v_r and q_r of X (*func:* `last_nonzero_singular_triplet`):

$$X = \sum_{\phi=1}^{r} \sigma_\phi v_\phi q_\phi^\top, \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

In [2]:

```python
def apply_A(A, M):
    return np.einsum("ijk,jk->i", A, M)

def adjoint_A(A, y):
    weighted_A = y[:, np.newaxis, np.newaxis] * A
    G = np.sum(weighted_A, axis=0)
    return 0.5 * (G + G.T)

def min_eigpair_symmetric(G):
    vals, vecs = np.linalg.eigh(G)
    idx = np.argmin(vals)
    return float(vals[idx]), vecs[:, idx]

def last_nonzero_singular_triplet(X, rtol=1e-10):
    """
    Computes the last non-zero singular value (σ_r) of a matrix X,
    along with its corresponding left and right singular vectors (v_r, q_r).

    Parameters:
        X (np.ndarray): Input matrix of shape (n, r). Can also be a vector (1D or 2D).
        rtol (float): Relative tolerance to determine whether a singular value is zero.

    Returns:
        sigma_r (float): The smallest non-zero singular value (σ_r).
        v_r (np.ndarray): Corresponding left singular vector (shape: (n,)).
        q_r (np.ndarray): Corresponding right singular vector (shape: (r,)).

    Raises:
        ValueError: If the matrix has rank 0 (i.e., all singular values are zero).
    """
    # Ensure X is at least 2D (e.g., convert a vector to column matrix if needed)
    X = np.atleast_2d(X)
```

```
    # Perform singular value decomposition: X = V S Q^T
    V, S, Qh = np.linalg.svd(X, full_matrices=False)

    # Determine the numerical rank by counting singular values > tolerance
    tol = rtol * S[0]  # tolerance is relative to the largest singular value
    rank = np.sum(S > tol)

    if rank == 0:
        raise ValueError("The input matrix has rank 0 — no non-zero singular values.")

    # Get the last non-zero singular value and its corresponding singular vectors
    sigma_r = S[rank - 1]          # Last non-zero singular value
    v_r = V[:, rank - 1]           # Corresponding left singular vector
    q_r = Qh[rank - 1, :]          # Corresponding right singular vector (from Q^T)

    return sigma_r, v_r, q_r
```

Functional Helpers for Matrix Sensing and Optimization

- Evaluate the objective function $h(X)$ in matrix space (*func:* `matrix_objective`):

$$h(X) = \frac{1}{2}\|\mathcal{A}(XX^\top - ZZ^\top)\|_2^2 = \frac{1}{2}\langle \mathcal{A}(XX^\top) - b, \mathcal{A}(XX^\top) - b\rangle,$$
$$\text{where } b = \mathcal{A}(ZZ^\top) = \mathcal{A}(M^*) \in \mathbb{R}^m.$$

- Compute the gradient $\nabla h(X)$ (*func:* `matrix_gradient`):

$$\nabla h(X) = 2\mathcal{A}^*\mathcal{A}(XX^\top - ZZ^\top)X = 2\sum_{i=1}^m \langle A_i, XX^\top - ZZ^\top\rangle A_i X = 2$$
$$\sum_{i=1}^m (\langle A_i, XX^\top\rangle - b_i)A_i X = 2GX \in \mathbb{R}^{n\times r},$$

  where the symmetric matrix $G \in \mathbb{R}^{n\times n}$ is defined as:

$$G = \mathcal{A}^*\mathcal{A}(XX^\top - ZZ^\top) = \sum_{i=1}^m \langle A_i, XX^\top - ZZ^\top\rangle A_i = \sum_{i=1}^m (\langle A_i, XX^\top\rangle - b_i)A_i.$$

- Run vanilla gradient descent (GD) in matrix space (*func:* `run_gd`):

$$X^{(t+1)} = X^{(t)} - \mathrm{lr}\cdot\nabla h(X^{(t)}).$$

- Estimate the RIP constant δ_p via random sampling (*func:* `estimate_delta_p`):

$$(1-\delta_p)\|XX^\top\|_F^2 \le \|\mathcal{A}(XX^\top)\|_2^2 \le (1+\delta_p)\|XX^\top\|_F^2.$$

In [3]:
```
def matrix_objective(A, X, b):
    M = X @ X.T
    y = apply_A(A, M)
    r = y - b
    return 0.5 * float(r @ r)

def matrix_gradient(A, X, b):
    M = X @ X.T
    y = apply_A(A, M)
    G_M = adjoint_A(A, y - b)
```

```python
    return 2.0 * (G_M @ X)

def run_gd(A, Z, b, X0, lr, steps, grad_tol):
    X = X0.copy()
    traj = [X.copy()]
    losses = [matrix_objective(A, X, b)]
    gradnorms = [float(np.linalg.norm(matrix_gradient(A, X, b), "fro"))]
    dists = [np.linalg.norm(X @ X.T - Z @ Z.T, "fro")]

    for _ in range(steps):
        nabla_h = matrix_gradient(A, X, b)
        X = X - lr * nabla_h

        gnorm = float(np.linalg.norm(matrix_gradient(A, X, b), "fro"))
        traj.append(X.copy())
        losses.append(matrix_objective(A, X, b))
        gradnorms.append(gnorm)
        dists.append(np.linalg.norm(X @ X.T - Z @ Z.T, "fro"))
        if gnorm <= grad_tol:
            break

    return traj, np.array(losses), np.array(gradnorms), np.array(dists)

def estimate_delta_p(A, p, num_samples=5000, seed=0, matrix_type='symmetric', normalize=
    """
    Estimate RIP constant delta_p for a linear operator A.

    Parameters:
    -----------
    A : np.ndarray
        Array of shape (m, n, n), representing m sensing matrices A_i of size n x n.
    p : int
        Target rank for low-rank matrix sampling.
    num_samples : int
        Number of random samples to estimate the RIP constant.
    seed : int
        Random seed for reproducibility.
    matrix_type : str
        Type of matrix to sample: 'general', 'symmetric', or 'psd'.
    normalize : bool
        Whether to normalize sampled matrices to unit Frobenius norm.

    Returns:
    --------
    delta_p : float
        Estimated RIP constant.
    """
    rng = np.random.default_rng(seed)
    m, n, _ = A.shape
    A_flat = A.reshape(m, -1)  # shape (m, n^2)

    r_min, r_max = float("inf"), float("-inf")

    for _ in range(num_samples):
        # --- Step 1: Sample a rank-p matrix M ---
        if matrix_type == 'psd':
            X = rng.standard_normal((n, p))
            M = X @ X.T  # PSD
        elif matrix_type == 'symmetric':
            X = rng.standard_normal((n, p))
            M = X @ rng.standard_normal((p, n))
            M = 0.5 * (M + M.T)  # Symmetric
        elif matrix_type == 'general':
            X = rng.standard_normal((n, p))
```

```
            Y = rng.standard_normal((n, p))
            M = X @ Y.T  # General rank-p
        else:
            raise ValueError(f"Invalid matrix_type: {matrix_type}")

        # --- Step 2: Normalize if needed ---
        fro_norm = np.linalg.norm(M, 'fro')
        if fro_norm == 0:
            continue
        if normalize:
            M = M / fro_norm

        # --- Step 3: Compute measurement: y = A(M) ---
        M_flat = M.reshape(-1)  # shape (n^2,)
        y = A_flat @ M_flat     # shape (m,)

        # --- Step 4: Compute squared measurement norm ---
        val = y @ y
        if normalize:
            val = val  # Already normalized
        else:
            val = val / (fro_norm ** 2)  # adjust for unnormalized M

        # --- Step 5: Track min and max ---
        r_min = min(r_min, val)
        r_max = max(r_max, val)

    # --- Step 6: Estimate delta_p ---
    delta_p = max(1.0 - r_min, r_max - 1.0)
    return float(delta_p)
```

Functional Helpers for Tensor Lifting

- Lift a vector $\mathrm{vec}(X)$ to tensor space (*func:* `elevate_vector`):

$$\mathbf{w} = \mathrm{vec}(X)^{\otimes l} \in \mathbb{R}^{(nr)^{\odot l}}, \quad \text{where } X \in \mathbb{R}^{n \times r}.$$

- Evaluate the lifted objective $h^l(\mathbf{w})$ in tensor space (*func:* `tensor_objective`):

$$h^l(\mathbf{w}) = \frac{1}{2} \| \langle \mathbf{A}^{\otimes l}, \langle \mathbf{P}(\mathbf{w}), \mathbf{P}(\mathbf{w}) \rangle_{2,4,\cdots,2l} \rangle_{2,3,5,6,\cdots,3l-1,3l} - b^{\otimes l} \|_F^2.$$

  Equivalently,

$$h^l(\mathbf{w}) = \frac{1}{2} \| \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle_{2,4,\cdots,2l} - \langle Z^{\otimes l}, Z^{\otimes l} \rangle_{2,4,\cdots,2l} \rangle_{2,3,5,6,\cdots,3l-1,3l} \|_F^2.$$

  To simplify notation, we omit contraction subscripts and expand the inner product

$$\langle \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle - \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle - \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle \rangle,$$

  yielding

$$= \underbrace{\langle\langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l}\rangle\rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l}\rangle\rangle\rangle}_{:=\mathrm{Awwww}} + \underbrace{\langle\langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l}\rangle\rangle, \langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l}\rangle\rangle\rangle}_{:=\mathrm{Azzzz}}$$
$$- 2\underbrace{\langle\langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l}\rangle\rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l}\rangle\rangle\rangle}_{:=\mathrm{Azzww}}.$$

A commonly reused term `A_topA` is precomputed as:

$$\mathbf{A}^*\mathbf{A} := \langle \mathbf{A}, \mathbf{A}\rangle_1 \in \mathbb{R}^{n\times n\times n\times n},$$

where $\mathbf{A}[i,:,:] = A_i \in \mathbb{R}^{n\times n}$.

- Precompute lifted ground truth term (*func:* `compute_Azzzz`):

$$\begin{aligned}\mathrm{Azzww} &= \langle\langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l}\rangle\rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l}\rangle\rangle\rangle \\ &= \langle\langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l}\rangle\rangle, \langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l}\rangle\rangle\rangle = \mathrm{Awwzz},\end{aligned}$$

which can be reused to avoid redundant computation in the lifted loss.

In [4]:
```python
def elevate_vector(w_in, level):
    """
    Elevates a 1D vector to a higher-dimensional tensor space.

    Parameters:
    ----------
    w_in : array-like
        Input vector of shape (n,).
    level : int
        The lifting level; the final tensor has dimensionality (odd number).

    Returns:
    -------
    lifted : array-like
        A high-dimensional lifted tensor.
    """
    if level == 0:
        return w_in
    w_in = w_in.reshape(-1)  # Flatten the input to ensure 1D
    einsum_w_args = list(chain(*[(w_in, (i,)) for i in range(level)]))
    lifted_tensor = jnp.einsum(*einsum_w_args)  # Perform Einstein summation
    return lifted_tensor

def tensor_objective(w, z_lifted, A_topA, Azzzz, level):
    """
    Computes the loss in a memory-efficient way.

    Parameters:
    ----------
    w : jnp.ndarray
        Current variable in the lifted space.
    z_lifted : jnp.ndarray
        Lifted version of the target vector z.
    A_topA : jnp.ndarray
        Precomputed tensor product of A with itself.
    Azzzz : jnp.ndarray
        Precomputed value of the lifted tensor z.
    level : int
        The lifting level.
```

```
    Returns:
    -------
    loss : float
        The computed loss.
    """
    # Build arguments for einsum
    args1 = []
    for i in range(level):
        args1 += [A_topA, (i * 4, i * 4 + 1, i * 4 + 2, i * 4 + 3)]

    args2 = [w, tuple(np.arange(level) * 4)]
    args3 = [w, tuple(np.arange(level) * 4 + 1)]
    args4 = [w, tuple(np.arange(level) * 4 + 2)]
    args5 = [w, tuple(np.arange(level) * 4 + 3)]

    args = args1 + args2 + args3 + args4 + args5

    # Compute Awwww
    Awwww = jnp.einsum(*args)

    # Rebuild args for Azzww
    args2 = [z_lifted, tuple(np.arange(level) * 4)]
    args3 = [z_lifted, tuple(np.arange(level) * 4 + 1)]

    args = args1 + args2 + args3 + args4 + args5

    Azzww = jnp.einsum(*args)

    # Compute loss
    loss = 0.5 * (Awwww + Azzzz - 2 * Azzww)
    return loss

def compute_Azzzz(A_topA, z_lifted, level):
    """
    Computes the lifted tensor Azzzz in a higher-dimensional space.

    Parameters:
    ----------
    A_topA : jnp.ndarray
        Tensor of shape (n, n, n, n).
    z_lifted : jnp.ndarray
        Lifted tensor of shape (level,).
    level : int
        The lifting level (must be odd).

    Returns:
    -------
    Azzzz : jnp.ndarray
        The lifted tensor in the higher-dimensional space.
    """
    # Compute args1
    args1 = []
    for i in range(level):
        args1 += [A_topA, (i * 4, i * 4 + 1, i * 4 + 2, i * 4 + 3)]

    # Compute args2, args3, args4, args5
    args2 = [z_lifted, tuple(np.arange(level) * 4)]
    args3 = [z_lifted, tuple(np.arange(level) * 4 + 1)]
    args4 = [z_lifted, tuple(np.arange(level) * 4 + 2)]
    args5 = [z_lifted, tuple(np.arange(level) * 4 + 3)]

    # Combine args and compute Azzzz
    args = args1 + args2 + args3 + args4 + args5
    return jnp.einsum(*args)
```

Load Hyper-parameters and MS Problem

In [5]:
```
# Load hyper-parameters
class Args:
    seed = 327  # Random seed
    r = 2  # Rank parameter r for RIP sampling (delta_2r*=1/2 is a sharp bound)
    delta_samples = 5000  # Number of RIP samples
    l = 3  # Lift order l
    rho = 0.1  # Escape step size in tensor space
    eta = 0.01  # Truncated PGD step size in tensor space
    t = 660  # Truncated PGD step numbers (Simulated)
    init_eps = 0.06  # Tiny init magnitude epsilon
    gd_lr = 0.01  # Matrix-space GD learning rate
    gd_steps_before_lift = 300  # GD steps before lifting
    gd_steps_after_collapse = 200  # GD steps after collapsing
    gd_gradtol = 1e-8  # Stopping threshold for ||grad||_F
args = Args()

# Load MS problem
def load_problem():
    n, m, r = 2, 3, 1
    A1 = np.array([[1, 0], [0, 0.5]])
    A2 = np.array([[0, np.sqrt(3)/2], [np.sqrt(3)/2, 0]])
    A3 = np.array([[0, 0], [0, np.sqrt(3)/2]])
    A = np.vstack([A1, A2, A3]).reshape((m, n, n))
    Z = np.array([[1.0], [0.0]])
    X_hat = np.array([[0.0], [1/np.sqrt(2)]])
    return n, m, r, A, Z, X_hat

n, m, r, A, Z, X_hat = load_problem()
Z = Z.astype(float)
M_star = Z @ Z.T
X_hat = X_hat.astype(float)
M_hat = X_hat @ X_hat.T
```

Compute the Escape Feasibility Score (EFS)

The Escape Feasibility Score is defined as:

$$\mathrm{EFS} := \underbrace{\frac{-\lambda_n}{\sigma_r^2(1+\delta_p)}}_{:=\mathrm{CSR}} + \underbrace{\frac{(\mathrm{daa\_tmp})^2}{2(1+\delta_p)^2}}_{:=\mathrm{DAA}},$$

where

$$\mathrm{daa\_tmp} := \sum_{i=1}^{m} \langle A_i, u_n v_r^\top + v_r u_n^\top \rangle \langle A_i, u_n u_n^\top \rangle.$$

If $\mathrm{EFS} > 1$, the current point $\hat{x}$ is escapeable via a single-step perturbation.

In [6]:
```
b = apply_A(A, M_star)
residual = apply_A(A, M_hat) - b
G = adjoint_A(A, residual)
lambda_min, u_n = min_eigpair_symmetric(G)
sigma_r, v_r, q_r = last_nonzero_singular_triplet(X_hat)
uvvu = np.outer(u_n, v_r) + np.outer(v_r, u_n)
uu = np.outer(u_n, u_n)
```

```
daa_tmp = np.sum(
    np.sum(A * uvvu, axis=(1, 2)) * np.sum(A * uu, axis=(1, 2))
)
delta_p = estimate_delta_p(A, p=args.r, num_samples=args.delta_samples, seed=args.seed,
                           matrix_type='symmetric', normalize=True)
CSR = (-lambda_min) / (sigma_r ** 2 * (1 + delta_p))
DAA = (daa_tmp ** 2) / (2 * (1 + delta_p) ** 2)
EFS = CSR + DAA

print("===== Single Step Escape =====")
print(f"n={n}, m={m}, r={r}, delta_p={delta_p:.6f}")
print(f"lambda_min={lambda_min:.6f}, sigma_r={sigma_r:.6f}, daa_tmp={daa_tmp:.6f}")
print(f"CSR={CSR:.6f}, DAA={DAA:.6f}, EFS={EFS:.6f}")
```

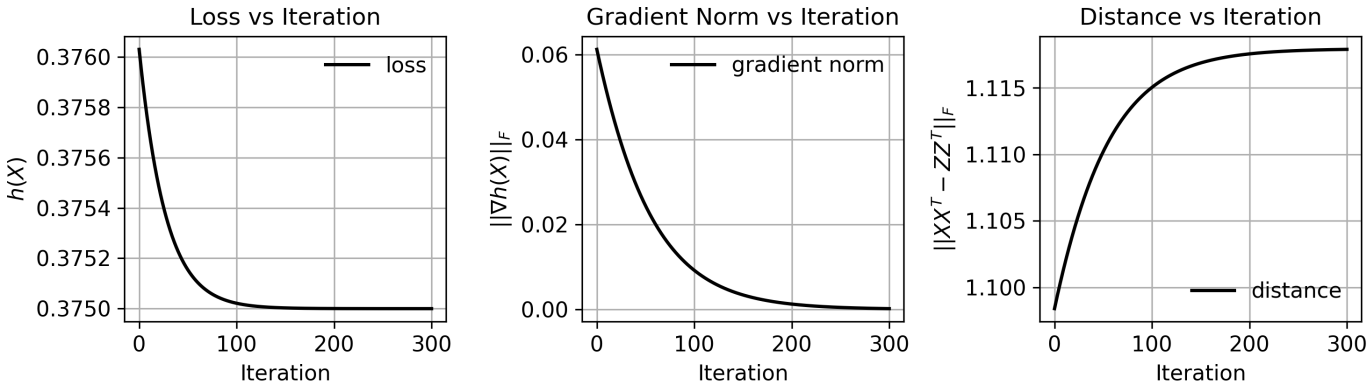
```
===== Single Step Escape =====
n=2, m=3, r=1, delta_p=0.500000
lambda_min=-0.750000, sigma_r=0.707107, daa_tmp=0.000000
CSR=1.000000, DAA=0.000000, EFS=1.000000
```

Run GD in Matrix Space (Pre-Escape Phase)

In [7]:

```
rng = np.random.default_rng(args.seed)
X0 = X_hat + args.init_eps * rng.normal(size=X_hat.shape)
traj_before, losses_before, gradnorms_before, dists_before = run_gd(
    A, Z, b, X0, args.gd_lr, args.gd_steps_before_lift, args.gd_gradtol)
X_local = traj_before[-1]
print(X_local)
print(len(traj_before))
print(len(losses_before))
print(len(gradnorms_before))
print(len(dists_before))

# Set up a 1x3 grid of subplots
fig1, axes1 = plt.subplots(1, 3, figsize=(10, 3))

ax1A = axes1[0]
ax1A.plot(range(args.gd_steps_before_lift+1), losses_before, color='black', label='loss'
ax1A.set_xlabel('Iteration', fontsize=11)
ax1A.set_ylabel(r'$h(X)$', fontsize=11)
ax1A.tick_params(axis='both', labelsize=11)
ax1A.set_title('Loss vs Iteration', fontsize=12)
ax1A.legend(frameon=False, fontsize=11)
ax1A.grid(True)

ax1B = axes1[1]
ax1B.plot(range(args.gd_steps_before_lift+1), gradnorms_before, color='black', label='gr
ax1B.set_xlabel('Iteration', fontsize=11)
ax1B.set_ylabel(r'$||\nabla h(X)||_F$', fontsize=11)
ax1B.tick_params(axis='both', labelsize=11)
ax1B.set_title('Gradient Norm vs Iteration', fontsize=12)
ax1B.legend(frameon=False, fontsize=11)
ax1B.grid(True)

ax1C = axes1[2]
ax1C.plot(range(args.gd_steps_before_lift+1), dists_before, color='black', label='distan
ax1C.set_xlabel('Iteration', fontsize=11)
ax1C.set_ylabel(r'$||XX^T - ZZ^T||_F$', fontsize=11)
ax1C.tick_params(axis='both', labelsize=11)
ax1C.set_title('Distance vs Iteration', fontsize=12)
ax1C.legend(frameon=False, fontsize=11)
ax1C.grid(True)

plt.tight_layout()
plt.show()
```

```
[[0.00839904]
 [0.70692475]]
301
301
301
301
```



Simulated Tensor Over-parameterization

We consider three base tensors in the lifted subspace S:

$$\begin{aligned} \text{a\_vec} := \text{vec}(\hat{X}), \quad \mathbf{a} &:= \text{vec}(\hat{X})^{\otimes l}, \\ \text{b\_vec} := \text{vec}(u_n q_r^\top), \quad \mathbf{b} &:= \text{vec}(u_n q_r^\top)^{\otimes l}, \\ \text{c\_vec} := \text{vec}(\mathcal{A}^*\mathcal{A}(u_n v_r^\top + v_r u_n^\top)\hat{X}), \quad \mathbf{c} &:= \text{vec}(\mathcal{A}^*\mathcal{A}(u_n v_r^\top + v_r u_n^\top)\hat{X})^{\otimes l}. \end{aligned}$$

We define the perturbed tensor (escape direction) as:

$$\mathbf{w}_{\text{escape}} := \mathbf{a} + \rho \cdot \mathbf{b}.$$

The lifted objective at the saddle point is denoted as:

$$h^l(\hat{\mathbf{w}}) = h^l(\mathbf{a}) =: \text{hl\_alpha\_a},$$

assuming $\tilde{\alpha}^{(t)} = 1$ during the truncated projected gradient descent (simulated) process.
The geometric sum used in coefficient updates is:

$$\sum_{\tau=0}^{t} \nu^\tau = \frac{1-\nu^{t+1}}{1-\nu}, \quad \text{where } \nu := 1 - \eta\lambda^l.$$

The dynamics of the lifted rank-1 tensors are:

$$\begin{aligned} \tilde{\alpha}^{(t)} = 1, \quad \text{a\_lift} = \tilde{\alpha}^{(t)} \cdot \mathbf{a}, \quad \text{hl\_alpha\_a} &= h^l(\text{a\_lift}), \\ \tilde{\beta}^{(t)} = \rho\nu^t, \quad \text{b\_lift} = \tilde{\beta}^{(t)} \cdot \mathbf{b}, \quad \text{hl\_beta\_b} &= h^l(\text{b\_lift}), \\ \tilde{\gamma}^{(t)} = -\eta\rho\frac{\sigma_r^l}{2^{l-1}}(\sum_{\tau=0}^{t-1}\nu^\tau), \quad \text{c\_lift} = \tilde{\gamma}^{(t)} \cdot \mathbf{c}, \quad \text{hl\_gamma\_c} &= h^l(\text{c\_lift}). \end{aligned}$$

- If $\|\text{c\_lift}\|_F \gg \max\{\|\text{a\_lift}\|_F, \|\text{b\_lift}\|_F\}$ and $\text{hl\_gamma\_c} < h^l(\hat{\mathbf{w}})$, then escape is successful via collapsing the c_lift term.
- If $\|\text{b\_lift}\|_F \gg \max\{\|\text{a\_lift}\|_F, \|\text{c\_lift}\|_F\}$ and $\text{hl\_beta\_b} < h^l(\hat{\mathbf{w}})$, then escape is successful via collapsing the b_lift term.

In [8]:

```
l = args.l
a_vec = X_hat.reshape(-1)
b_vec = np.outer(u_n, q_r).reshape(-1)
c_vec = (adjoint_A(A, apply_A(A, uvvu)) @ X_hat).reshape(-1)
lam_pow = lambda_min ** l
nu = 1.0 - args.eta * lam_pow

def geom_sum(t):
    if abs(nu - 1.0) < 1e-12:
        return float(t + 1)
    return float((1.0 - (nu ** (t + 1))) / (1.0 - nu))

# lifted constants
A_topA = jnp.einsum("ijk,ilm->jklm", A, A)
z_lifted = elevate_vector(Z, level=l)
Azzzz = compute_Azzzz(A_topA, z_lifted, level=l)

T = args.t
alpha_t = 1.0  # does not change with t
a_vec_lift = elevate_vector(a_vec, level=l)
b_vec_lift = elevate_vector(b_vec, level=l)
c_vec_lift = elevate_vector(c_vec, level=l)
w_escape = a_vec_lift + args.eta * b_vec_lift
a_lift = alpha_t * a_vec_lift
hl_alpha_a = tensor_objective(a_lift, z_lifted, A_topA, Azzzz, level=l)
hl_w_escape = tensor_objective(w_escape, z_lifted, A_topA, Azzzz, level=l)

#hl_beta_b_list = []
#hl_gamma_c_list = []
hl_w_list = []
a_lift_list = []
b_lift_list = []
c_lift_list = []
for tt in tqdm(range(T)):
    beta_t = args.rho * (nu ** (tt + 1))
    gamma_t = -args.eta * args.rho * ((sigma_r ** l) / (2 ** (l - 1))) * geom_sum(tt)
    b_lift = beta_t * b_vec_lift
    c_lift = gamma_t * c_vec_lift

    hl_beta_b = tensor_objective(b_lift, z_lifted, A_topA, Azzzz, level=l)
    hl_gamma_c = tensor_objective(c_lift, z_lifted, A_topA, Azzzz, level=l)
    hl_w_t = tensor_objective(a_lift + b_lift + c_lift, z_lifted, A_topA, Azzzz, level=l
    #hl_beta_b_list.append(hl_beta_b)
    #hl_gamma_c_list.append(hl_gamma_c)
    hl_w_list.append(hl_w_t)

    a_lift_norm = jnp.linalg.norm(a_lift)
    b_lift_norm = jnp.linalg.norm(b_lift)
    c_lift_norm = jnp.linalg.norm(c_lift)
    a_lift_list.append(a_lift_norm)
    b_lift_list.append(b_lift_norm)
    c_lift_list.append(c_lift_norm)

print("===== Lifted objective values =====")
print(f"h^l(alpha_a) = {hl_alpha_a:.10f}")
print(f"h^l(beta_b) = {hl_beta_b:.10f}")
print(f"h^l(gamma_c) = {hl_gamma_c:.10f}")
print(f"h^l(w_hat)    = {hl_alpha_a:.10f}")
print(f"h^l(w_escape) = {hl_w_escape:.10f}")
print(f"h(w^(t+1))  = {hl_w_t:.10f}")
print(f"||alpha*a||={a_lift_list[-1]:.6f}, "
      f"||beta*b||={b_lift_list[-1]:.6f}, "
      f"||gamma*c||={c_lift_list[-1]:.6f}")
```

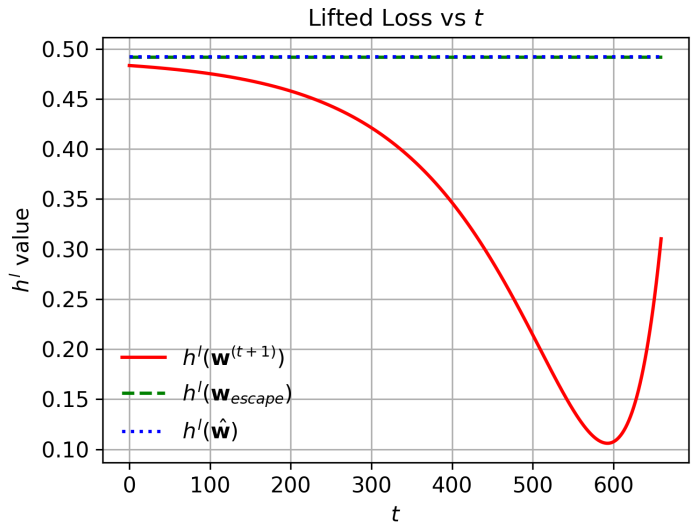
```
# Collapse selection (relaxed dominance)
collapse_ok = False
collapse_mode = None
X_check = None
if hl_w_t < hl_alpha_a:
    if (c_lift_norm > b_lift_norm) and (c_lift_norm > a_lift_norm):
        collapse_ok = True
        collapse_mode = "gamma"
        #hl_vals = hl_gamma_c_list
    elif (b_lift_norm > c_lift_norm) and (b_lift_norm > a_lift_norm):
        collapse_ok = True
        collapse_mode = "beta"
        #hl_vals = hl_beta_b_list
    else:
        print("Simulating Tensor Over-parameterization Failed!")
else:
    print("collapse_ok is", collapse_ok)
```

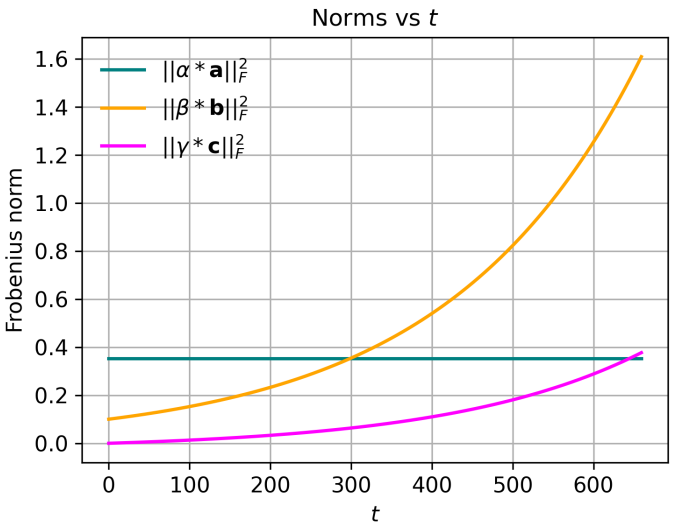
```
100%|██████████████████████████████████████████| 660/660 [02:37<00:00,  4.18it/s]
===== Lifted objective values =====
h^l(alpha_a) = 0.4921875000
h^l(beta_b) = 1.2649078369
h^l(gamma_c) = 0.3677262068
h^l(w_hat)     = 0.4921875000
h^l(w_escape) = 0.4920996130
h(w^(t+1))  = 0.3102724552
||alpha*a||=0.353553, ||beta*b||=1.609515, ||gamma*c||=0.377379
```

Visualization of Simulated Tensor Over-parameterization

In [9]:

```
# Set up a 1x2 grid of subplots
fig2, axes2 = plt.subplots(1, 2, figsize=(10, 4))

ax2D = axes2[0]
ax2D.plot(range(T), hl_w_list, color='red', label=r'$h^l(\mathbf{w}^{(t+1)})$', linewidt
ax2D.hlines(hl_w_escape, xmin=0, xmax=args.t,
            linestyle='--', color='green', label=r'$h^l(\mathbf{w}_{escape})$', linewidt
ax2D.hlines(hl_alpha_a, xmin=0, xmax=args.t,
            linestyle=':', color='blue', label=r'$h^l(\hat{\mathbf{w}})$', linewidth=1.7
ax2D.set_xlabel(r'$t$', fontsize=11)
ax2D.set_ylabel(r'$h^l$ value', fontsize=11)
ax2D.tick_params(axis='both', labelsize=11)
ax2D.set_title('Lifted Loss vs $t$', fontsize=12)
ax2D.legend(frameon=False, loc='lower left', fontsize=11)
ax2D.grid(True)

ax2E = axes2[1]
ax2E.plot(range(T), a_lift_list, color='teal', label=r'$||\alpha * \mathbf{a}||_F^2$', l
ax2E.plot(range(T), b_lift_list, color='orange', label=r'$||\beta * \mathbf{b}||_F^2$',
ax2E.plot(range(T), c_lift_list, color='magenta', label=r'$||\gamma * \mathbf{c}||_F^2$'
ax2E.set_xlabel(r'$t$', fontsize=11)
ax2E.set_ylabel('Frobenius norm', fontsize=11)
ax2E.tick_params(axis='both', labelsize=11)
ax2E.set_title('Norms vs $t$', fontsize=12)
ax2E.legend(frameon=False, fontsize=11)
ax2E.grid(True)

plt.tight_layout()
plt.savefig('exp1_fig1.pdf', dpi=300, format='pdf')
plt.show()
```





Collapse from Tensor Space via Closed-Form Solution

- If collapsing $\mathrm{c\_lift}$ term:

$$\check{X} = -(-\tilde{\gamma}^{(t)})^{\frac{1}{l}} \cdot \mathcal{A}^*\mathcal{A}(u_n v_r^\top + v_r u_n^\top)\hat{X} \in \mathbb{R}^{n\times r};$$

- If collapsing $\mathrm{b\_lift}$ term:

$$\check{X} = (\tilde{\beta}^{(t)})^{\frac{1}{l}} \cdot (u_n q_r^\top) \in \mathbb{R}^{n\times r}.$$

As a result, $\check{X}$ lies in matrix space and represents a point that successfully escapes the attraction basin of $\hat{X}$.

Note: Since $\check{X}$ admits a closed-form expression, **there is no need to perform actual tensor lifting** — the procedure is purely simulated.

In [10]:
```
print("collapse_mode is", collapse_mode)
if collapse_ok:
    if collapse_mode == "gamma":
        scale = np.sign(gamma_t) * (abs(gamma_t) ** (1.0 / l)) if gamma_t != 0 else 0.0
        X_check = (scale * c_vec).reshape(n, r)
    elif collapse_mode == "beta":
        scale = np.sign(beta_t) * (abs(beta_t) ** (1.0 / l)) if beta_t != 0 else 0.0
        X_check = (scale * b_vec).reshape(n, r)
```

```
collapse_mode is beta
```

Run GD in Matrix Space (Post-Escape Phase)

In [11]:
```
# If collapse performed, run GD from X_check
traj_after = []
losses_after = []
gradnorms_after = []
dists_after = []
#losses_full = list(losses_before)
#gradnorms_full = list(gradnorms_before)
#dists_full = list(dists_before)

if X_check is not None:
    traj_after, losses_after, gradnorms_after, dists_after = run_gd(
        A, Z, b, X_check, args.gd_lr, args.gd_steps_after_collapse, args.gd_gradtol)
    #losses_full.extend(losses_after)
```

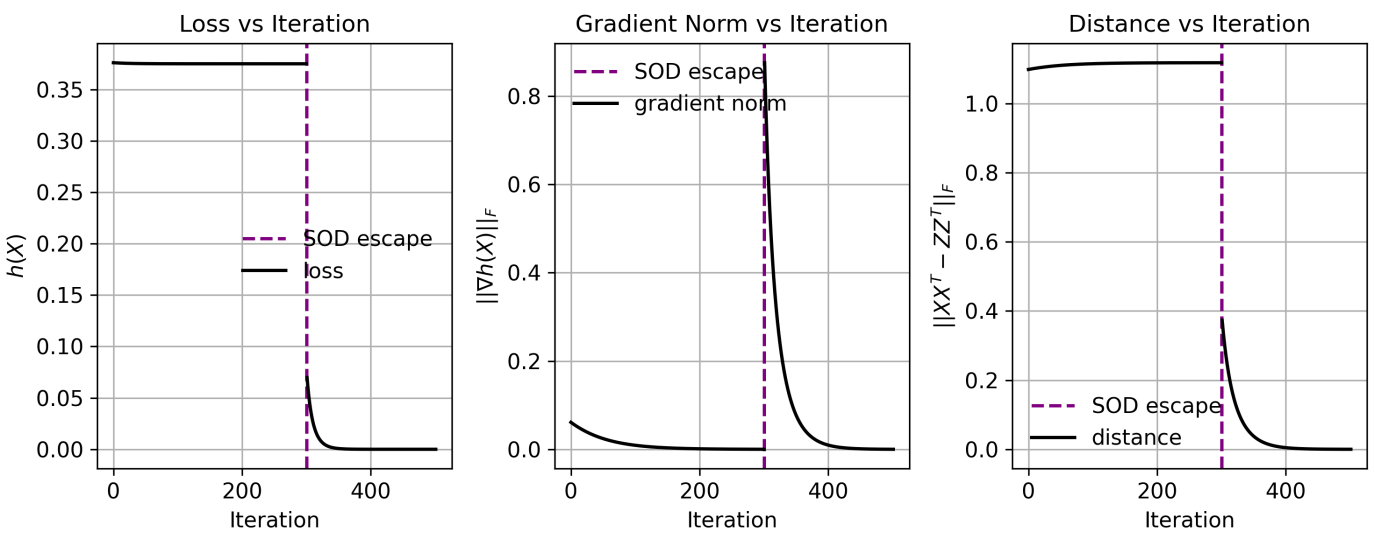
```
    #gradnorms_full.extend(gradnorms_after)
    #dists_full.extend(dists_after)

print(len(traj_after))
print(len(losses_after))
print(len(gradnorms_after))
print(len(dists_after))
```

```
201
201
201
201
```

Full Procedure Visualization

In [12]:

```
# Set up a 1x3 grid of subplots
fig3, axes3 = plt.subplots(1, 3, figsize=(10, 4))

ax3A = axes3[0]
ax3A.axvline(len(losses_before) - 1, linestyle='--', color='purple', label='SOD escape',
ax3A.plot(range(args.gd_steps_before_lift + 1),
          losses_before, color='black', label='loss', linewidth=1.7)
if len(losses_after) > 0:
    ax3A.plot(range(args.gd_steps_before_lift + 1, args.gd_steps_before_lift + args.gd_s
              losses_after, color='black', linewidth=1.7)
ax3A.set_xlabel('Iteration', fontsize=11)
ax3A.set_ylabel(r'$h(X)$', fontsize=11)
ax3A.tick_params(axis='both', labelsize=11)
ax3A.set_title('Loss vs Iteration', fontsize=12)
ax3A.legend(frameon=False, fontsize=11)
ax3A.grid(True)

ax3B = axes3[1]
ax3B.axvline(len(gradnorms_before) - 1, linestyle='--', color='purple', label='SOD escap
ax3B.plot(range(args.gd_steps_before_lift + 1),
          gradnorms_before, color='black', label='gradient norm', linewidth=1.7)
if len(gradnorms_after) > 0:
    ax3B.plot(range(args.gd_steps_before_lift + 1, args.gd_steps_before_lift + args.gd_s
              gradnorms_after, color='black', linewidth=1.7)
ax3B.set_xlabel('Iteration', fontsize=11)
ax3B.set_ylabel(r'$||\nabla h(X)||_F$', fontsize=11)
ax3B.tick_params(axis='both', labelsize=11)
ax3B.set_title('Gradient Norm vs Iteration', fontsize=12)
ax3B.legend(frameon=False, loc='upper left', fontsize=11)
ax3B.grid(True)

ax3C = axes3[2]
ax3C.axvline(len(dists_before) - 1, linestyle='--', color='purple', label='SOD escape',
ax3C.plot(range(args.gd_steps_before_lift + 1),
          dists_before, color='black', label='distance', linewidth=1.7)
if len(dists_after) > 0:
    ax3C.plot(range(args.gd_steps_before_lift + 1, args.gd_steps_before_lift + args.gd_s
              dists_after, color='black', linewidth=1.7)
ax3C.set_xlabel('Iteration', fontsize=11)
ax3C.set_ylabel(r'$||XX^T - ZZ^T||_F$', fontsize=11)
ax3C.tick_params(axis='both', labelsize=11)
ax3C.set_title('Distance vs Iteration', fontsize=12)
ax3C.legend(frameon=False, fontsize=11)
ax3C.grid(True)

plt.tight_layout()
plt.savefig('exp1_fig2.pdf', dpi=300, format='pdf')
plt.show()
```



In [13]:

```
# Plot 3d Trajectory
fig4 = plt.figure()
ax3d = fig4.add_subplot(1, 1, 1, projection='3d')
x1_vals = np.linspace(-1.1, 1.1, 100)
x2_vals = np.linspace(-1.1, 1.1, 100)
X1, X2 = np.meshgrid(x1_vals, x2_vals)

def surface_loss(x1, x2):
    return (x1**2 - 1 + 0.5 * x2**2)**2 + (np.sqrt(3) * x1 * x2)**2 + ((np.sqrt(3)/2) *

Z_vals = surface_loss(X1, X2)
ax3d.plot_surface(X1, X2, Z_vals, alpha=0.16, cmap='viridis', edgecolor='none')

traj_np = np.array([X.reshape(-1) for X in traj_before])
zs = np.array([surface_loss(x1, x2) for x1, x2 in traj_np])
ax3d.plot(traj_np[:,0], traj_np[:,1], zs, color='blue', label='GD before SOD escape', li

ax3d.scatter(X_local[0, 0], X_local[1, 0], surface_loss(*X_local.reshape(-1)),
             color='blue', s=60, label=r'local minimum $\hat{X}$', marker='o')
ax3d.scatter(Z[0, 0], Z[1, 0], surface_loss(*Z.reshape(-1)),
             color='red', s=80, label=r'ground truth $M^*$', marker='*')

if X_check is not None:
    ax3d.scatter(X_check[0, 0], X_check[1, 0], surface_loss(*X_check.reshape(-1)),
                 color='green', s=80, label=r'escaped point $\check{X}$', marker='o')
    start = X_local.reshape(-1)
    end = X_check.reshape(-1)
    dz = surface_loss(*end) - surface_loss(*start)
    ax3d.quiver(start[0], start[1], surface_loss(*start), *(end - start), dz, arrow_leng
                label='SOD escape direction', color='purple', linewidth=1.7)
    traj_after_np = np.array([X.reshape(-1) for X in traj_after]) if len(traj_after) > 0
    if traj_after_np.size > 0:
        za = np.array([surface_loss(x1, x2) for x1, x2 in traj_after_np])
        ax3d.plot(traj_after_np[:,0], traj_after_np[:,1], za,
                  color='green', label='GD after SOD escape', linewidth=1.7)

ax3d.set_xlabel(r'$x_1$', fontsize=11)
ax3d.set_ylabel(r'$x_2$', fontsize=11)
ax3d.set_zlabel(r'$h(x)$', fontsize=11)
ax3d.tick_params(axis='x', labelsize=11)
ax3d.tick_params(axis='y', labelsize=11)
ax3d.tick_params(axis='z', labelsize=11)
ax3d.set_title('Surface + GD Trajectory (with SOD escape)', fontsize=12)
ax3d.legend(frameon=False, ncol=3, loc='upper center', fontsize=11)
ax3d.view_init(elev=39, azim=-58)
```

```
plt.tight_layout()
plt.savefig('exp1_fig3.pdf', dpi=300, format='pdf')
plt.show()
```